

Sistemas Operativos

Práctico 2: Tareas, concurrencia y planificación de cpu

Ejercicio 1: Dado el programa *counter.c*, compilarlo en el ejecutable *counter*, crear el archivo *counter.dat* el cual debe contener el valor inicial 0 (como texto) y realizar los siguientes experimentos:

1. Ejecutar el comando `./counter ; ./counter` y analizar la salida.

La ejecución secuencial del algoritmo 2 veces, sumó hasta 2000. Y es correcto, ya que el algoritmo tiene un for y suma el valor que escribe en el archivo en 1, 1000 veces. Al ejecutarse 2 veces suma hasta 2000.

2. Reinicializar el valor en el archivo de datos y disparar dos o más procesos concurrentes (`./counter & ./counter`).

El valor ahora me dio como resultado: 1578.

3. Analizar y dar una explicación del resultado obtenido en *counter.dat*.

El resultado que me dio es porque el algoritmo de counter está accediendo a un recurso (en este caso el *counter.dat*), al ejecutar 2 procesos concurrentes, puede ocurrir por ejemplo: que un proceso haya leído lo que hay en el archivo, y le paso el control al 2do proceso, esté lee, incrementa, guarda y se reanuda el primer proceso, el cual como ya leyó tiene un valor erróneo, ya que el proceso 2 ya lo modifico.

4. Solucionar el problema de concurrencia usando la función `flock()` de UNIX.

```
int pid = getpid(), i;
int fd = open("counter.dat", O_RDWR);

for (i=0; i<1000; i++) {
    char s[N];
    int n;

    flock(fd, LOCK_EX);
    lseek(fd, 0, SEEK_SET);
    read(fd, s, N);
    n = atoi(s);
    sprintf(s, "%05d", n+1);
    lseek(fd, 0, SEEK_SET); // rewind
    write(fd, s, strlen(s));
    fsync(fd);
    flock(fd, LOCK_UN);
}
close(fd);
return 0;
```

Ejercicio 2: ¿Cómo se representa el estado de ejecución completo de una tarea que no es RUNNING?

- Cuando una tarea no está en estado RUNNING (ej. Está RUNNABLE, WAITING o TERMINATED), toda la información necesaria para reanudarla más tarde queda almacenada en el Task (`struct task`) y en su saved context (`struct context`).

```
struct task {
    int          tid;          // ID de la tarea
    char         name[TASK_NAME_LEN]; // nombre para debugging
    int          state;        // UNUSED, RUNNABLE, WAITING, etc.
    int          exit_code;    // si TERMINATED, el código de salida
    struct context ctx;        // ① contexto de CPU guardado
    uint8        kstack[PAGE_SIZE]; // ② pila de kernel propia
    uint64        sleep_ticks; // contador para sleep()
    int           cpu_id;      // CPU donde estaba corriendo (-1 si ninguna)
    void*         wait_condition; // condición por la que espera (si WAITING)
    spinlock      lock;       // para sincronizar tablas de tareas
};
```

- **ctx (struct context)**
 - Guarda los registros callee-saved y el `ra/sp` tal como estaban justo antes de ceder la CPU.
 - Es lo **mínimo** que el planificador (scheduler) necesita para restaurar la ejecución.
- Cada tarea tiene su propia pila de kernel. Cuando se restaura `ctx.sp`, el puntero de pila volverá a apuntar al tope correcto de esta `kstack`.
- **state**: indica si la tarea está RUNNABLE, WAITING, TERMINATED, ...
- **sleep_ticks** y **wait_condition**: si está bloqueada, para qué exactamente (timer, recurso, etc.).
- **cpu_id**: en qué CPU estaba corriendo (o -1 si ninguna).

Con todo esto el planificador puede detener, reordenar y reanudar tareas de forma transparente, retomando exactamente su estado de ejecución.

Ejercicio 3: Proponer una implementación de `fork()` para tareas en edos-steps (código del mini-kernel dado). En particular describir cómo se logra que en el padre retorne el *pid* del hijo y en el hijo el valor 0.

El `fork` es un call-system, para poder realizarlo debemos agregarlo a la función `trap`, añadiendo un nuevo **case: SYSCALL**.

Como sabemos el `fork` debe hacer lo siguiente:

- Crear un nuevo proceso (task), que copie el contexto (pila y registros) del que lo ejecutó (padre) pasando a ser este nuevo proceso su hijo.
- Cuando llamamos a la instrucción `fork()`, está debe devolver el `pid` del hijo (`pid > 0`).
- Y el hijo, devuelve el `pid` del padre.

Como sería su implementación?:

- Además de agregar el caso que dijimos anteriormente en la función `trap`, debemos copiarle al nuevo `task_hijo`, el contexto del padre (`struct context ctx`), a su vez también debemos copiar la pila que tiene el proceso padre y que sea igual para el proceso hijo.
- Luego en la función del `fork`, debemos agregarle el `pid` al hijo, en nuestro caso tenemos una variable que va incrementando a medida que vamos teniendo más procesos que es `last_tid` (`task_hijo->tid = ++last_tid`).
- ACÁ ME FALTA CÓMO HACER QUE EL HIJO DEVUELVE EL PID DEL PADRE (`pid = 0`)
- Una vez realizado esto, ya el scheduler puede elegir si ejecutar el hijo, el padre u otra.

Ejercicio 4: Describa el flujo de ejecución cuando un proceso es interrumpido por el *timer* hasta retornar de la interrupción.

Cuando un proceso es interrumpido por el timer lo que sucede es que Nosotros configuramos el kernel para que cuando se produzca una excepción el **HARDWARE** ejecute el trap de **MACHINE MODE**.

1. El CSR (Control and Status Register) `mtvec` (Machine Trap-Vector Base-Address Register) tiene la dirección de inicio del manejador de traps en modo **MACHINE**.
2. El hardware guarda el **PC** (en `sepc`) y el **cause** (causa por la que se generó la interrupción)

Se lo dimos en el boot antes de empezar el kernel.

3. Dentro de `m_trap` en machine mode:
 - 1) Reserva el espacio necesario en la pila para salvar registros `a0–a7`, los salva ya que la función que llama `next_timer_interrupt(hartid)` puede usar estos registros y no queremos perder su valor original cuando retorne.

- 2) Proteger el valor de ra (return address) en a7 para saber exactamente donde se estaba ejecutando el proceso antes de que se haya producido la interrupción porque la llamada a la función que se hará la modifica.
- 3) Llama a `next_timer_interrupt(hartid)` para reprogramar el siguiente tick.
- 4) Restaura ra, con el valor que guardo en a7.
- 5) Restaura los registros de a0-a7 y limpia la pila del espacio que había reservado para salvarlos.
- 6) Señala al Supervisor Mode que hay un timer interrupt pendiente, osea que Setea el bit SSIP en SIP para delegar a Supervisor
- 7) Ejecuta `mret`, lo que sucede acá es que:
 - Cambia el modo de M a S (según MSTATUS y delegaciones).
 - Restaura el PC desde `sepc`.
 - Detecta que SSIP está activo y salta al vector `stvec`, invocando la rutina `s_trap` en Supervisor Mode.

4. `mret` hace que cambie de machine mode a modo supervisor.

5. Carga el PC desde `sepc`, que contiene la instrucción interrumpida en S-mode (o el punto justo después de la última instrucción de supervisor).

6. El hardware detecta el bit de SSIP y dispara el trap en S-mode.

7. **Entrada a `s_trap`** en modo Supervisor. Ahí el código en ensamblador:

- Reserva espacio en pila y salva **todo** el contexto de usuario (ra, gp, t0-t6, a0-a7, s0-s11 y sp) en el orden segun el struct `trap_frame`.
- Hace `mv a0, sp; call trap;` para invocar la rutina C `trap(tf)` pasando el puntero al trap frame.

8. Dentro de la rutina `C trap()`

En `trap(struct trap_frame *tf)` tu kernel:

- Lee `scause`, `sepc`, `sstatus` para saber qué ocurrió y dónde.
- Atiende el caso `TIMER_INTERRUPT`: incrementa el contador de ticks, hace `ack_timer_interrupt()` para limpiar `sip.SSIP`, y eventualmente llama a `yield()` si el quantum expiró.
- Restaura `sepc` y `sstatus` para volver al punto interrumpido.

- Retorna de `trap()`, luego el ensamblador de `s_trap` hace `sret`, que restaura los registros salvados y ejecuta `sret`, subiéndote de nuevo al código de usuario o supervisor donde ibas.

<https://chatgpt.com/share/6827c2e8-7ba4-8003-94d9-5fec8c61cfcd>

Ejercicio 5: ¿Cuál es la diferencia entre el trapframe y el contexto salvado de cada tarea? Especificar cómo se usa cada uno de ellos.

- El trapframe: Se usa al entrar y salir de una interrupción (ya sea por el timer, alguna excepción, syscall, etc). Esta estructura guarda todos los registros (ra, gp, t0-t6, a0-a7, s0-s11, sp). Lo hace para que el manejador del trap pueda inspeccionarlos (argumentos, return address, punteros, etc.), y después poder volver al código que había sido interrumpido en el momento.
Esta solo durante la ejecución del trap handler y que después, con `sret`, la CPU restaura esa foto completa y vuelve al punto exacto interrumpido.
- El contexto salvado en cada tarea, es al hacer `yield`, `suspend` o al cambiar de tarea en el scheduler. Guarda los 'calle-save' (s0-s11).
Además también guarda el return address (para saber exactamente qué instrucción estaba ejecutando de la tarea) y sp (stack pointer, para saber donde es la ubicación del toma de la pila de esa tarea).
Lo usamos dentro de cada struct task mientras la tarea está pausada, y se restaura con `context_switch()` para reanudarla desde donde quedó.

Ejercicio 6: Suponiendo que los procesos usarán la cpu de la forma

Proceso	Duración de la próxima ráfaga
P1	8
P2	3
P3	5

Determinar el tiempo de espera promedio para los algoritmos FCFS (FIFO) y Shortest Job First (SJF).

Veamos primero el tiempo de espera de **FCFS (FIFO)**: Este agrega el nuevo proceso al último de una cola y solo puede ejecutarlo si llego a estar primero y todos los anteriores procesos que estaban delante de él terminaron.

El orden de ejecución será: P1 → P2 → P3

P1 ingresa a la cola (primero ya que la cola está vacía).

- COMIENZO: 0
- ESPERA: 0

P2 esta luego de P1:

- Comienzo = 8
- Espera = 8

P3 esta luego de P2:

- Comienzo = 8 + 3
- Espera = 8 + 3

La espera total va ser $8 + 11 = 19$

El promedio de espera va ser $19/3 = 6.33$

Ahora veamos Shortest Job First (SJF):
supongamos que:

- $\alpha = 0.5$ (peso mitad al dato reciente y mitad al histórico)
- Estimación inicial $T_n = 5$ (un valor arbitrario ej. el promedio de ráfagas)

Entonces aplicamos la fórmula

$$T_{n+1} = \alpha t_n + (1 - \alpha) * T_n$$

Y luego elegimos el proceso que en la formula nos de el menor tiempo.

Lo hacemos:

P1:

$$T_0 = 5$$

$$t_0 = 8$$

$$\text{Formula: } T1 = 0,5*8 + (1-0,5) * 5 = 6,5$$

P2:

$$T_0 = 5$$

$$t_0 = 3$$

$$\text{Formula: } T1 = 0,5*3 + (1-0,5) * 5 = 4$$

P3:

$$T_0 = 5$$

$$t_0 = 5$$

$$\text{Formula: } T1 = 0,5*3 + (1-0,5) * 5 = 5$$

Si el scheduler usara **shortest-estimated-burst-first**, ordenaría:

1. **P2** con T=4.0
2. **P3** con T=5.0
3. **P1** con T=6.5

Ejercicio 7: Realizar un experimento de planificación de cpu en GNU-Linux con un proceso de uso intensivo de cpu. Puede crear un script de shell como se muestra abajo:

```
$ cat > infinite.sh
#!/bin/bash
for(( ; ; ))
do
    continue
done <ctrl-d>
$ chmod 744 infinite.sh
```

1. Ejecutarlo normalmente y en otra terminal ver el uso de cpu con el comando top.
2. Correr una instancia con menor prioridad (ej: nice -n 15 bash ./infinite.sh) y observar los procesos nuevamente con top.
3. Cambiar la prioridad de una de las instancias en ejecución (ver el comando renice).
4. Hacer que un proceso ejecute con mayor prioridad que la de un proceso normal (ejemplo: con un valor de niceness de -10).

Ejercicio 8: Implementar semáforos en edos-steps. Probarlos creando tareas que representen el problema del productor-consumidor.

Para implementar los semáforos tenemos que tener en cuenta su estructura:

```
struct semaphore {
    int value      //contador de cantidad de tareas que lo pueden usar
    spinlock lock //
}
```

Después tenemos las funciones:

- sem_init(s, value): Inicializa el valor del semáforo s.
- sem_wait(s): Si el valor de s está en cero, suspende al invocante. Sino, decrementa el valor del semáforo s.
- sem_signal(s): Incrementa el valor de s y despierta a los threads o procesos bloqueados en s

☒ ~~LISTO EN PRACTICÓ 2 SEMÁFOROS.~~